Análisis y visualización de algoritmos de ordenamiento

Germán A. Holguín L.

Noviembre, 2026

Introducción

El ordenamiento de datos permite estudiar y comparar diferentes estrategias de diseño al gorítmico, así como evaluar su eficiencia en términos de recursos computacionales. Aunque diversos algoritmos pueden producir un mismo resultado, sus requerimientos de tiempo y memoria pueden variar considerablemente en función del tamaño y las características de los datos de entrada.

Entre los algoritmos clásicos de ordenamiento se encuentra *Bubble Sort*, basado en la compa ración de elementos adyacentes y su reorganización progresiva mediante un proceso iterativo. Por otra parte, *Quicksort* utiliza la estrategia de dividir y conquistar, mediante la partición de los datos alrededor de un elemento denominado pivote y la resolución recursiva de los subproblemas resultantes. Estas diferencias permiten estudiar cómo las estrategias de diseño algorítmico influyen en la eficiencia computacional de las soluciones.

A partir del estudio de estos algoritmos, se desarrollará una aplicación en Python con interfaz gráfica de usuario (GUI), utilizando agentes de inteligencia artificial como herramientas de apoyo al desarrollo. La aplicación permitirá comparar experimentalmente el comportamiento de ambos algoritmos, contrastar los resultados obtenidos con su análisis teórico y visualizar interactivamente el proceso de ordenamiento.

1. Análisis y comparación de algoritmos de ordenamiento

Desarrolle una aplicación de software en Python, con GUI, que permita implementar, analizar y comparar el comportamiento computacional de los algoritmos *Bubble Sort* y *Quicksort*, utilizando arreglos unidimensionales de números, que pueden ser tanto enteros como reales. Para su desarrollo, deberá utilizar un agente de inteligencia artificial, como Codex, Cursor, Antigravity o Claude Code, entre otros.

El desarrollo deberá comprender las siguientes actividades:

1. Análisis formal de los algoritmos. Estudie ambos algoritmos, considerando *Bubble Sort* en su implementación iterativa y *Quicksort* en su implementación recursiva. Para cada uno, establezca y justifique su correctitud, demuestre su terminación y determine su complejidad temporal y espacial en los casos pertinentes; lo anterior, de acuerdo a los contenidos vistos en clase.

   Presente explícitamente el procedimiento matemático y algorítmico utilizado para obtener dichos resultados. No será suficiente indicar únicamente las expresiones de complejidad asintótica.

2. Implementación computacional. Implemente ambos algoritmos en Python sin utilizar funciones o métodos de ordenamiento predefinidos. Verifique su funcionamiento mediante casos de prueba que incluyan arreglos de números enteros y reales, elementos repetidos, valores negativos y diferentes configuraciones iniciales.

3. Comparación experimental. Desarrolle un módulo que permita ejecutar ambos algo ritmos sobre conjuntos de datos equivalentes, considerando diferentes tamaños de entrada y condiciones iniciales.

   Diseñe una metodología experimental que contemple diferentes tamaños de entrada y configuraciones de los datos, incluyendo arreglos aleatorios, previamente ordenados e in versamente ordenados. La cantidad y los tamaños de los conjuntos de datos, así como el número de repeticiones de cada experimento, deberán ser seleccionados y justificados por el estudiante, de manera que permitan caracterizar el crecimiento del tiempo de ejecución y contrastarlo con el comportamiento teórico de ambos algoritmos.

   Las mediciones deberán excluir las operaciones de generación de datos y actualización de la interfaz gráfica.

4. Visualización e interpretación de resultados. Incorpore en la interfaz gráfica herra mientas para configurar los experimentos y representar los tiempos de ejecución de ambos algoritmos en función del tamaño de entrada.

   Contraste los resultados experimentales con las complejidades temporales obtenidas me diante el análisis formal. Interprete las tendencias observadas, explique las posibles dife rencias entre el comportamiento teórico y experimental e identifique las condiciones bajo las cuales cada algoritmo presenta ventajas o limitaciones computacionales.

La estrategia de selección del pivote en *Quicksort* será definida y justificada por el estudiante, considerando sus implicaciones sobre el comportamiento del algoritmo.

2. Visualización interactiva de algoritmos

Seleccione uno de los algoritmos implementados en el punto anterior y desarrolle un módulo de animación que permita visualizar, de manera interactiva y secuencial, las operaciones realizadas durante el proceso de ordenamiento.

Para esta actividad, deberá utilizar un agente de inteligencia artificial diferente del empleado en la primera sección.

La solución deberá satisfacer los siguientes requisitos:

1. Representación visual. Diseñe una animación que represente gráficamente los elementos de un arreglo de tamaño reducido, utilizando barras, bloques u otros recursos visuales que permitan identificar sus valores y posiciones.

2. Visualización de operaciones. Represente secuencialmente las operaciones relevantes del algoritmo seleccionado, incluyendo las comparaciones, los intercambios y, cuando co rresponda, la selección del pivote y las particiones del arreglo.

   La animación deberá reflejar fielmente las operaciones realizadas por el algoritmo imple mentado.

3. Interacción con el usuario. Incorpore controles que permitan ingresar o generar los datos de entrada, iniciar, pausar, reanudar y reiniciar la animación, así como modificar su velocidad de ejecución.

4. Restricciones tecnológicas. La animación deberá desarrollarse mediante herramientas de la biblioteca estándar de Python, utilizando Tkinter o Turtle.

   Se permite utilizar Matplotlib para la representación de gráficas experimentales, pero no bibliotecas adicionales para interfaces gráficas, animaciones o videojuegos, como PyQt, Pygame o similares.

El módulo de animación deberá integrarse en la aplicación desarrollada en el primer punto, reutilizando la implementación del algoritmo seleccionado.

3. Entregables

El trabajo deberá entregarse con los siguientes componentes:

1. Código fuente. Proyecto completo en Python, con la aplicación funcional, sus módulos de comparación y animación, y las instrucciones necesarias para su ejecución.

2. Informe técnico. Documento en formato PDF que contenga el análisis formal de ambos algoritmos, incluyendo la justificación de correctitud, terminación y complejidad temporal y espacial, con sus respectivos desarrollos matemáticos.

3. Resultados experimentales. Descripción de la metodología experimental, casos de prueba, gráficas comparativas, análisis de los resultados e interpretación de las diferencias entre el comportamiento teórico y experimental.

4. Documentación del desarrollo asistido por inteligencia artificial. Registro resu mido de los agentes empleados, las instrucciones principales y las intervenciones realizadas por el estudiante durante el desarrollo y la validación.